



Universidade do Minho

Programação Concorrente

Licenciatura em Ciências da Computação

Grupo 18

Oliver Favero Teixeira a102506

Rita Mota Machado a102508

Guilherme Fernandes Duarte a

Hélder Simão Lima Fernandes Pereira a

Índice

1	Introdução	3
2	Desenvolvimento do trabalho	3
2.1	Autenticação	3
2.2	Lobby e inicialização das partidas	3
2.3	Espaço e Física de Movimento.....	4
2.4	Sistema de Colisões e Repulsão	4
2.5	Sistema de Capturas.....	4
2.6	Modificadores	4
2.7	Pontuação	5
3	Cliente e Servidor.....	5
3.1	Estrutura do Cliente	5
3.2	Estrutura do Servidor	6
4	Conclusão	6

1 Introdução

Nesta unidade curricular, foi proposto a implementação de um jogo multijogador em tempo real. Inspirado em mecânicas de jogos de sobrevivência de arena (como o agar.io), o principal objetivo deste projeto consiste em aplicar os paradigmas de programação concorrente lecionados.

Este projeto encontra-se dividido em dois grandes blocos: um servidor programado em Erlang, responsável pela física, regras e gestão de salas; e um cliente gráfico em Java (utilizando a biblioteca Processing), este, encarregue de processar o input do utilizador, renderizar a interface gráfica e comunicar com o servidor através de sockets TCP.

2 Desenvolvimento do trabalho

2.1 Autenticação

O nosso sistema permite aos utilizadores de efetuarem Login, Register e Unregister. Pagar garantir a persistência de dados, o estado das contas de utilizador não reside apenas em memória. As funções `load_accounts` e `save_accounts` operam sobre o ficheiro `accounts.txt`, assegurando que o mapa de utilizadores registre permanentemente as contas criadas e as respetivas senhas.

2.2 Lobby e inicialização das partidas

O processo `lobby_loop` atua como uma sala de espera que agrupa jogadores num sistema de fila. Assim que a fila chegue a 4 jogadores ou 3 jogadores após o timeout de 10 segundos, o servidor remove esses jogadores da fila e coloca-os para um novo processo independente através da função `init_game`. Assim, garantimos que o sistema pode instanciar um máximo de 4 jogadores em simultâneo sem bloquear a receção de novos jogadores.

2.3 Espaço e Física de Movimento

A movimentação dos jogadores é calculada pelo servidor e enviada para o cliente. A aceleração e o ângulo de rotação dependem da massa do jogador, utilizando as constantes Força e Torque divididas pela massa atual. Isto introduz uma mecânica de equilíbrio, sendo jogadores com maior massa movimentam-se e rodam de forma mais lenta do que jogadores com menores. Foi implementado também um fator de Fricção que decai progressivamente a velocidade, evitando deslizes infinitos.

2.4 Sistema de Colisões e Repulsão

Em cenários que os avatares colidem (distância entre os centros menores que a soma dos raios) mas não existe uma diferença de massa substancial que justifique uma captura (menos de 1.2x de diferença de massa), o motor do jogo aplica uma função de repulsa mútua (`aplicar_repulsao_global`). Essa força empurra cada jogador no vetor oposto da colisão por uma distância correspondente a metade da sobreposição ($\text{Overlap} * 0.5$), evitando que as esferas se sobreponham graficamente.

2.5 Sistema de Capturas

A mecânica central do jogo assenta na absorção da massa adversária. Uma captura somente ocorre se o jogador atacante tiver no mínimo 1.2x de massa superior a massa do outro jogador e se a distância entre eles permitir cobrir significativamente a esfera menor. Quando a captura é efetuada, o atacante adquire um quarto da massa do jogador menor e soma um ponto ao Score. Já o jogador que foi capturado, por sua vez, reaparece numa localização aleatória do mapa de volta a sua massa mínima.

2.6 Modificadores

O mapa mantém dinamicamente os modificadores instanciados que estes podem ser comestíveis ou venenosos. Para prevenir situações onde um recém-rencido não consiga evoluir, a função `garantir_pequeno` avalia continuamente a massa do menor jogador em jogo, gerando, se necessário, um objeto ainda menor que ele para garantir o seu crescimento inicial.

2.7 Pontuação

O estado global das pontuações mais altas é feito pelo processo autónomo `top_scores_loop`. Ao finalizar a partida, os pontos obtidos são enviados de forma assíncrona para este processo. A função de ranking efetua a ordenação da tabela, descartando duplicatas e retendo apenas os 10 maiores resultados históricos.

3 Cliente e Servidor

3.1 Estrutura do Cliente

O cliente Java se baseia num sistema multi-thread, onde temos uma thread principal encarregue da renderização gráfica (`draw()`, executando a 60 frames por segundo) e uma thread paralela dedicada à rede focada na leitura do socket (`ConnectionServer`).

Uma vez que ambas acedem e modificam as mesmas estruturas globais, tivemos de impedir as race conditions que pudessem resultar em exceções como `ConcurrentModificationException`.

Também, de acordo com o padrão estudado de exclusão mútua, utilizamos um `ReentrantLock`. Tanto o método `draw()` como o interpretador de rede adquirem o lock (`lock.lock()`) antes de operarem nas estruturas e utilizam mandatoriamente um bloco `try{...} finally {lock.unlock();}` para assegurar que a secção crítica é libertada mesmo mediante a falhas inesperadas. Além disso, foi adicionada uma variável de condição (`Condition estadoMudou`) para suportar a transição eficiente entre o Lobby e o Jogo, implementando mecanismos como o `await()` e `signalAll()`.

3.2 Estrutura do Servidor

A abordagem no servidor difere muito no cliente devido ser com a linguagem Erlang. Em oposição ao modelo de memória partilhada com locks do Java, o Erlang opera sob o Modelo de Atores. Não existe estado partilhado (cada processo possui a sua própria heap) e os problemas clássicos de concorrência são evitados. Toda a comunicação processa-se unicamente através de passagem de mensagem assíncronas entre processos ultra-leves e isolados (ex: comunicação entre GameRoom, LobbyPid e TopScoresPid).

4 Conclusão

O desenvolvimento deste jogo em rede serviu para consolidar as bases lecionadas ao longo da UC. Consideramos também que todos os objetivos foram alcançados. No Java/Processing, experienciámos a complexidade de evitar interferências entre threads. Já com Erlang, aprendemos como desenvolver um sistema desenhado desde a sua raiz para escalar processos isolados através de passagem de mensagens. Por fim, este trabalho permitiu-nos aprender boas práticas para no futura aplicar os conceitos lecionados nesta UC.